

Final Project Report

Team 8

Yusen Han, Yifei Li, Linfei Yuan, Zijin Zhou

December 18, 2025

1 Introduction (Project Scope)

This project is a pick-and-place challenge where the Panda arm must autonomously detect, pick, and stack both stationary and moving blocks onto the team-colored goal platform to build the tallest tower and maximize score. The overall task therefore includes two subproblems: (i) a reliable static pick-and-place pipeline and (ii) a dynamic pipeline that handles blocks in motion, each under the same game constraints.

Matches run under strict competition constraints (randomized block poses, fixed match duration, and a shared environment; Figure 1). The system must run fully autonomously during a match (no manual adjustments), and it must operate safely using only the provided interface (a single `ArmController` with “safe” motion commands).

Perception is performed via AprilTag-based detections from an end-effector camera. Simulation provides near-ground-truth information for rapid iteration, and hardware gets real camera detections, which motivates robustness and sim-to-real validation.

Scoring depends on successfully placing both static and dynamic blocks on the goal platform, with points determined by block type (Value) and stacking height (Altitude). As a result, the project scope includes not only reliable manipulation and safety, but also a practical game strategy that prioritizes high-scoring actions under time and uncertainty constraints.

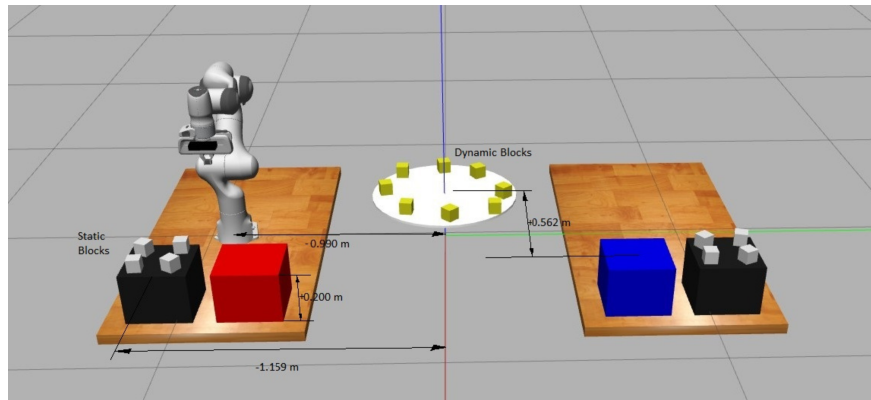


Figure 1: Project Competition Environment in Simulation

2 Method

2.1 System Components

2.1.1 Forward Kinematics

To systematically describe the geometry of the robot and compute its forward kinematics, we adopted the standard Denavit-Hartenberg (DH) convention. The geometric dimensions were determined from the robot datasheet, and the resulting DH parameters were derived based on the coordinate frame assignments shown in Figure 2

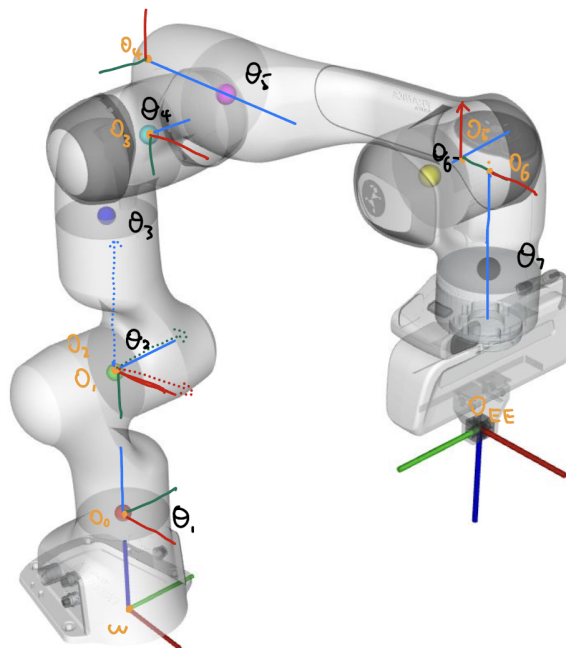


Figure 2: DH Coordinate Frames, R,G,B corresponds to x, y, z axes

Link	a_i	α_i	d_i	θ_i
1	0	$-\pi/2$	0.192	θ_1
2	0	$\pi/2$	0	θ_2
3	0.0825	$-\pi/2$	$0.195 + 0.021$	θ_3
4	-0.0825	$-\pi/2$	0	$-\theta_4$
5	0	$\pi/2$	$-(0.259 + 0.125)$	$-\theta_5$
6	0.088	$-\pi/2$	0	$-\theta_6$
7	0	0	$0.051 + 0.159$	$\theta_7 - \pi/4$

Table 1: DH Parameters

2.1.2 Inverse Kinematics

To find the joint angles needed to reach a specific end-effector pose, we implemented a numerical inverse kinematics solver using gradient descent. This approach treats IK as an optimization problem as following:

Given a desired end effector pose $H = \begin{bmatrix} R & o \\ 0 & 1 \end{bmatrix}$ and the robot manipulator with 7 joints, find the joint angles $q = [q_1, \dots, q_7]$ such that minimized the error $e = T_n^0(\mathbf{q}) - H$:

$$\min_{q \in \mathbb{R}^7} f(q) = e = T_n^0(\mathbf{q}) - H \quad (1)$$

subject to joint limits $q_{\min} \leq q \leq q_{\max}$

The solver uses an iterative gradient descent method to converge on a solution. In each iteration, the algorithm performs the following steps:

1. **Jacobian:** Calculate the 6×7 Jacobian matrix J for the current joint configuration
2. **End-Effector Goal:** Compute the change in joint angles Δq to reduce the end-effector error. We utilized the Jacobian Pseudoinverse method, which solves the least-squares problem:

$$\min_{\Delta q} ||J\Delta q - \xi|| \quad (2)$$

This yields the minimum-norm solution:

$$\Delta q = -\alpha J^+ \xi \quad (3)$$

where α is the descent step size and ξ is the spatial velocity error vector

3. **Joint Centering:** To maintain desirable configurations, we applied a secondary task of joint centering in the null space of the Jacobian.
 - Compute the null-space projector: $N = I - J^+ J$
 - Calculate the joint centering velocity b to drive joints toward the center of their range
 - The final update combined both tasks:

$$dq_{total} = dq_{primary} + N \cdot b \quad (4)$$

4. **Update and Convergence:** The joint angles are updated ($q \leftarrow q + \alpha \cdot dq_{total}$) until the error falls below the specified tolerance or the maximum iteration count is reached.

2.1.3 Computer Vision

Block detection is performed using the perception module provided by the course infrastructure. Each block has AprilTags on all its faces, and a calibrated camera mounted on the robot's wrist is used to estimate the block pose relative to the camera frame via OpenCV-based AprilTag detection.

Detected block poses are accessed through the provided `ObjectDetector` interface. The known transformation between the camera and end-effector from 2.1.1 allows these poses to be expressed in the robot base or world frames as needed.

While simulation data is effectively ground truth, real-world detections can be noisy. To mitigate this, we measured and validated the physical camera offsets during lab calibration, and assumed a fixed z -height for all blocks, since they rest on a surface of known height at the start of the game.

2.2 Static Block

The manipulation pipeline is implemented as a Finite State Machine, coordinating the high-level logic with low-level command execution.

The process for static block pick and place is divided into four stages: detection, approach, grasping, and stacking.

2.2.1 Detection

The robot starts by scanning the static platform to find static blocks.

The robot is first moved to a pre-defined scan configuration that the field of view include all 4 static blocks. Calls the computer vision module (2.1.3) and then computes the pose of all visible blocks in the robot's base frame using FK (2.1.1).

Instead of re-scanning before picking every block, we aggregates valid block poses into a list. This allows the robot to iterate through the detected blocks, reducing cycle time by eliminating redundant scanning.

2.2.2 Approach & Descent

Once a target block is selected:

1. **Target Pose Calculation:** The grasp pose is computed for a top-down approach that aligns the gripper with the block.

Given the detected block rotation matrix $R_{block} = [\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3]$:

- (a) *Identify Vertical Axis:* The block axis most aligned with $\mathbf{z}_{world}$ is identified to distinguish the top face from the sides:

$$k = \operatorname{argmax}_{i \in \{1,2,3\}} |\mathbf{c}_i \cdot \mathbf{z}_{world}|$$

- (b) *Select Grasp Axis:* We then selects the axis $\mathbf{v}^*$ that best aligns with the world (robot) x-axis $\mathbf{x}_{world}$ from the remaining two. This ensures gripper yaw angle to be consistent across grasps to avoid awkward wrist configurations and keep the arm in a high-manipulability workspace.

$$\mathbf{v}^* = \operatorname{argmax}_{\mathbf{v} \in \{\mathbf{c}_j\}_{j \neq k}} |\mathbf{v} \cdot \mathbf{x}_{base}|$$

- (c) *Construct Rotation Matrix:* The final target rotation R_{target} is constructed with the z-axis pointing vertically down for a top-grasp:

$$\mathbf{z}_{target} = [0, 0, -1]^T$$

$$\mathbf{y}_{target} = \mathbf{z}_{target} \times \mathbf{x}_{target}$$

$$R_{target} = [\mathbf{x}_{target}, \mathbf{y}_{target}, \mathbf{z}_{target}]$$

2. **Pre-Grasp:** To prevent collisions with other blocks, the robot first moves to a PRE-GRASP waypoint located vertically above the target. The position P_{pre} is defined as:

$$P_{pre} = P_{target} + \begin{bmatrix} 0 \\ 0 \\ 0.15 \end{bmatrix} \text{ m}$$

3. **Descent:** From the **PRE-GRASP** position, the arm performs a vertical descent. The target z-height is calculated using the known platform height $h_{platform}$ and a hardware offset δ_{grasp} to prevent impact with the surface and mitigate noises from the computer vision system.

$$z_{grasp} = h_{platform} + \delta_{grasp}$$

2.2.3 Grasping

1. The gripper is actuated to close with a predefined position and gripping force.
2. **Validation:** A validation function monitors the gripper width w . The grasp is considered successful if the width remains non-zero for a period of time:

$$w_{final} > \epsilon_{min_width}$$

If the gripper fully closes $w \approx 0$, we identify the failure and move on to the next target

2.2.4 Stacking

1. **Height Calculation:** The placement height is calculated based on the current number of stacked blocks. The target height z_{drop} is calculated as:

$$z_{drop} = h_{platform} + (n_{blocks} \cdot h_{block}) + \delta_{safety}$$

where n_{blocks} is the current count of blocks stacked, h_{block} is block height, and δ_{safety} is a small buffer.

2. **Retraction:** After the gripper opens to release the block, the arm executes a vertical lift motion. This prevents the gripper fingers from accidentally knocking over the newly stacked tower.

2.3 Dynamic Block

Our approach to grasping dynamic blocks on a rotating turntable employs three modules: Detection, Prediction, and Grasp & Stack. The core of the algorithm lies in utilizing a known kinematic model of the turntable to predict the object's pose after a specified prediction time, combined with geometric projection to plan the optimal grasping pose. The Detection module and Grasp & Stack module are identical to those used in the Static Block Methodology. The detailed methodology is described below.

2.3.1 Detection Module

The robot arm first moves to the observation configuration q_{obs} . Using the camera system and the CV module, the object's pose is acquired. The pose of the object in the world coordinate frame is calculated via the homogeneous transformation chain.

2.3.2 Prediction Module

To account for system latency and robot execution time, the Prediction Module establishes a turntable motion model to predict the object's state at time t_{pred} .

Let the turntable center be C . The turntable rotates with a constant angular velocity ω . A feedforward prediction time Δt is defined. During this interval, the rotation angle θ is:

$$\theta = \omega \cdot \Delta t \quad (5)$$

The corresponding rotation matrix about the Z-axis, $R_z(\theta)$, is defined as:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Let the object's current position be P_{curr} . The predicted position P_{pred} is calculated as:

$$P_{pred} = R_z(\theta)(P_{curr} - C) + C \quad (7)$$

Simultaneously, as the object rotates rigidly with the turntable, its orientation (rotation matrix) R_{obj} must also be updated:

$$R_{pred} = R_z(\theta)R_{obj} \quad (8)$$

2.3.3 Target Selection

Through repeated experiments, an ‘‘Intercept Window’’ was defined to enhance grasping success rates. The grasping sequence is triggered only when the object's predicted X-coordinate, x_{pred} , falls within the following ranges:

- **Red Team:** $[-0.11, -0.06]$
- **Blue Team:** $[0.06, 0.11]$

This mechanism ensures that the robot operates within the optimal workspace where Inverse Kinematics (IK) solutions are stable.

2.3.4 Grasp Pose Planning

To ensure grasp stability, the target pose T_{target} must satisfy two constraints:

- **Vertical Constraint:** The end-effector Z-axis must point strictly downwards, i.e., $\mathbf{z}_{target} = [0, 0, -1]^T$.
- **Orientation Alignment:** Under the vertical constraint, the end-effector X-axis should align as closely as possible with the object's X-axis (following the object's rotation).

Our algorithm employs a Gram-Schmidt process to construct the target rotation matrix. Let the object's current X-axis direction be $\mathbf{x}_{block}$. First, project $\mathbf{x}_{block}$ onto the horizontal plane perpendicular to $\mathbf{z}_{target}$ to obtain the new X-axis vector $\mathbf{x}_{new}$:

$$\mathbf{x}' = \mathbf{x}_{block} - (\mathbf{x}_{block} \cdot \mathbf{z}_{target})\mathbf{z}_{target} \quad (9)$$

$$\mathbf{x}_{new} = \frac{\mathbf{x}'}{\|\mathbf{x}'\|} \quad (10)$$

Next, calculate the Y-axis via cross product to construct the orthogonal rotation matrix:

$$\mathbf{y}_{new} = \mathbf{z}_{target} \times \mathbf{x}_{new} \quad (11)$$

$$R_{target} = [\mathbf{x}_{new} \quad \mathbf{y}_{new} \quad \mathbf{z}_{target}] \quad (12)$$

The target joint angles q_{target} are computed using a numerical Inverse Kinematics algorithm with Jacobian Pseudo-Inverse, including joint centering optimization for the final joint.

Experimental results indicated that the grasping success rate is significantly higher when the gripper aligns its closing direction with the robot's y-axis rather than the x-axis. Since the gripper is initially aligned with the x-axis at the observation configuration, specific logic was implemented to reorient the final joint q_7 .

First, the angular difference between the target and current joint angle, $\Delta q_7 = q_{target,7} - q_{current,7}$, is normalized to the range $[-\pi/2, \pi/2]$. The algorithm then checks if the magnitude of this difference satisfies $|\Delta q_7| \leq \pi/4$. This condition indicates that the gripper is currently oriented closer to the suboptimal x-axis alignment. To correct this, a $\pi/2$ offset is applied:

$$q_{target,7} \leftarrow \begin{cases} q_{target,7} - \pi/2 & \text{if } \Delta q_7 > 0 \\ q_{target,7} + \pi/2 & \text{if } \Delta q_7 \leq 0 \end{cases} \quad (13)$$

This adjustment forces the gripper to rotate by 90° , ensuring it aligns with the optimal y-axis for grasping.

Additionally, to prevent excessive computation time, the IK_pseudo algorithm is limited to a maximum of 100 iterations. If a target configuration is not solved within this limit, the algorithm abandons the current dynamic block and resets to detect a new target.

2.4 Overall Game Strategy

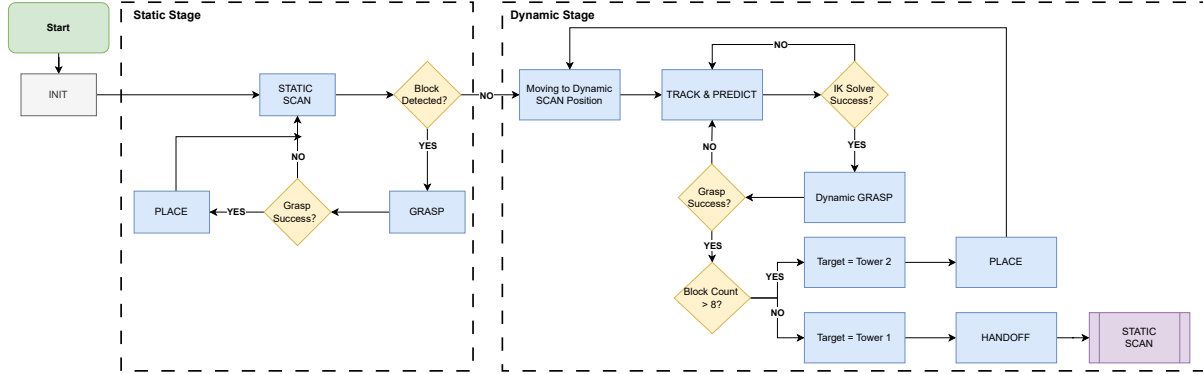


Figure 3: Flow Chart of FSM.

2.5 Code Structure

Our code is split into task-level logic in `lab/final/` and reusable kinematics utilities in `lib/` from previous labs. In `lab/final/`, `final.py` is the entry point that initializes ROS, loads team/config settings, constructs the main components, and runs the control loop; `fsm.py` implements the high-level finite-state machine and state transitions (scan $\rightarrow$ pick $\rightarrow$ place $\rightarrow$ recovery/dynamic) by calling perception and motion primitives; `driver.py` wraps the provided arm controller and our FK/IK implementations (from `lib/`) to provide motion/gripper primitives and robot state (e.g., `T_base_ee`); `vision.py` wraps the provided object detector and converts camera-frame detections into base-frame block poses using the hand-eye transform plus calibrated offsets with basic filtering/safety checks; and `config.py` centralizes tunable parameters (scan pose, offsets, thresholds, geometry), which are produced/validated using the interactive calibration script `calibrate.py`.

3 Evaluation

To ensure both system reliability and operational performance, we adopted a comprehensive "Sim-to-Real" evaluation strategy. This approach allowed us to validate high-level logic and control algorithms in a risk-free environment before addressing physical uncertainties on the hardware. The evaluation was conducted in two distinct phases: (1) Simulation Phase: The primary objective in the Gazebo simulation was to verify Logic Integrity. We used the noise-free environment to validate the modular integration between the Control, Vision, and FSM subsystems. (2) Hardware Phase: The hardware evaluation focused on bridging the "Reality Gap". This phase addressed real-world constraints such as sensor noise, mechanical latency, and friction. Our testing prioritized Parameter Tuning and validating the Dynamic Prediction algorithms against the physical turntable's movement.

3.1 Subcomponent Evaluation

3.1.1 Perception

The vision system was evaluated on hardware to ensure precise block localization. The results are summarized below in Table 2. The testing protocol involved recording static frames to quantify sensor noise and performing physical grasps to measure the offset calibration error.

Table 2: Evaluation Results: Perception Subsystem

Metric	Env.	Procedure	Result
Noise Analysis (<i>Precision</i>)	Hardware	Record 20 frames of vision data with a static block. Calculate the Standard Deviation of the detected coordinates.	1.28 cm (Target: < 0.1 cm)
Offset Calibration (<i>Accuracy</i>)	Hardware	Execute grasps at 4 known reference positions. Measure the Physical Offset between the gripper's center and the block's center.	0.57 cm (Target: < 0.5 cm)

3.1.2 Motion Control

The motion planning and execution subsystem was evaluated through stress testing and precision measurements. We assessed the robustness of the Inverse Kinematics (IK) solver by executing repetitive cycles in both simulation and hardware to identify potential singularities or failures. Additionally, we measured the vertical alignment error of stacked blocks to evaluate placement precision and tracked the success rate of grasp attempts in static and dynamic scenarios. The experimental setup and metrics are outlined in Table 3.

3.1.3 Dynamic Prediction

Evaluating the dynamic prediction subsystem required a qualitative approach due to the continuous motion of the turntable. Instead of measuring absolute coordinate errors, we categorized the outcome of dynamic grasp attempts into four distinct quality levels, ranging from perfect alignment

Table 3: Evaluation Results: Motion Control Subsystem

Metric	Env.	Procedure	Result	
			Static	Dynamic
IK Solver Success Rate	Sim.	Execute multiple cycles (30 for sim and 10 for hardware) to stress test the solver.	100% (Target: > 95%)	92% (Target: > 85%)
	Hard.		98% (Target: > 95%)	83% (Target: > 85%)
Placement Precision	Sim.	Stack 5 static blocks. Measure alignment error of the tower.	~ 0 cm (Ref: Near-zero)	
	Hard.		0.42 cm (Target: < 0.5 cm)	
Grasp Success Rate	Sim.	Execute 15 grasp attempts in each environment.	100% (Target: $\sim 100\%$)	95% (Target: > 94%)
	Hard.		96% (Target: > 95%)	92% (Target: > 85%)

to failure (as shown in Table 4). This method allows us to assess the effectiveness of the interception algorithm under real-time constraints.

Table 4: Evaluation Results: Dynamic Prediction Subsystem

Metric	Env.	Procedure	Result
Grasp Quality	Sim.	Categorize the outcome of 20 dynamic grasp attempts into 4 levels: 3: Perfect / Centered 2: Off-center but Safe 1: Edge / Risky 0: Failure	97% (Score 2-3) (Target: > 90%)
	Hard.		88% (Score 2-3) (Target: > 80%)

3.2 Integrated System Performance (Overall)

As shown in Table 5, the final stage of evaluation focused on the integrated system’s performance regarding robustness, efficiency, and task completion. We conducted continuous block runs to measure recovery rates from forced failures, average cycle times for static and dynamic blocks, and the maximum achievable tower height. These metrics assess the system’s ability to handle long-term operation and complex stacking tasks. Our final system performance and operational outcomes are demonstrated in the following video: Final Project Outcome Video.

4 Analysis

In the transition from simulation to reality (Sim-to-Real), the performance of the perception system serves as the primary factor influencing overall operational precision. According to the noise analysis in Table 2 (Perception Subsystem Evaluation), the standard deviation of coordinates detected by the perception subsystem reached 1.28 cm, which represents a significant gap from the preset target

Table 5: Evaluation Results: Integrated System

Metric	Env.	Procedure	Result	
			Static	Dynamic
Recovery Rate (<i>Robustness</i>)	Sim.	During a 20-block run, count successful Retries after a failed grasp.	100% (System never crashes)	
	Hard.		98% (Target: 100%)	
Cycle Time (<i>Efficiency</i>)	Sim.	Measure average time from block detection to stack completion.	15s	53s
	Hard.		16s (Target: < 15s)	42s (Target: < 25s)
Tower Height (<i>Result</i>)	Sim.	Stack blocks until tower collapses or reaches workspace limits.	Tower 1: 8 Blocks (4 Stat. + 4 Dyn.) Tower 2: 4 Blocks (Dynamic)	
	Hard.		Tower 1: 8 Blocks (4 Stat. + 4 Dyn.) Tower 2: 3 Blocks (Dynamic)	

of less than 0.1 cm. This higher-than-expected noise level directly increases the risk of grasping failure, making it difficult for the system to achieve perfect alignment solely through visual feedback in a physical environment. To address this deviation, our strategy includes a reserved window for offset adjustments, allowing for tuning to compensate for visual positioning inaccuracies and hardware variations. Consequently, the physical deviation measured in the offset calibration test was 0.57 cm, which is closely aligned with the target of less than 0.5 cm.

The stability of the motion control subsystem exhibited distinct performance differences between static and dynamic tasks. Based on the data in Table 3 (Motion Control Subsystem Evaluation), the Inverse Kinematics (IK) solver achieved a success rate of 98% in the hardware environment for static targets, meeting the operational requirements. However, this rate dropped to 83% in dynamic scenarios. This decline reflects a bottleneck in solver efficiency and convergence when processing real-time coordinate transformations generated by the rotating turntable. This limitation directly impacted the hardware dynamic grasp success rate, which decreased from 95% in simulation to 90% in reality. To mitigate this in dynamic environments, we incorporated tunable parameters for prediction time and detection zone thresholds, ensuring the algorithm completes planning within the optimal grasping window.

The dynamic prediction capability for turntable targets is essential for achieving complex stacking. Table 4 (Dynamic Prediction Subsystem Evaluation) shows that the proportion of high-quality hardware grasps (scores of 2–3) was 85%. While this exceeds the 80% target, it represents a regression from the 95% success rate observed in simulation. Due to physical latency and prediction errors, the end-effector often grasps the block at a "non-ideal center" position. In the context of dynamic stacking, such eccentric grasps reduce the structural stability of the tower, eventually leading to collapse. To counter this, we updated our strategy: after picking a dynamic block, the robot first places it on a scoring plate. It then performs a precise static grasp to pick the block up again, ensuring that dynamic blocks can be stacked securely at higher levels.

Based on the final performance of the integrated system, our robustness-oriented design effectively offset the errors inherent in perception and prediction. According to Table 5 (Integrated System Evaluation), the system achieved a recovery success rate of 98% in the hardware environment, validating the robustness of the retry mechanism against stochastic hardware failures. Regarding operational efficiency, the cycle time for static blocks was 16 seconds, and the dynamic cycle time was 22 seconds, both of which are closely aligned with the preset performance targets

of 15 seconds and 25 seconds, respectively. On physical hardware, we successfully reached a mixed stacking height of 8 layers and achieved the goal of stacking two separate towers.

In summary, after conducting multiple performance evaluations, we have quantified the impact of the Sim-to-Real gap on our algorithms and updated our grasping strategies to be more robust and better suited for real-world applications. Specifically, we determined that prior to hardware deployment, individual robots require parameter tuning to compensate for visual offsets and to optimize prediction time and detection intervals for dynamic targets. By refining the strategy to avoid tower instability caused by low-quality dynamic grasps, we ultimately achieved a hardware outcome of stacking 7 blocks (4 static and 3 dynamic) and a second tower with 1 dynamic block within 5 minutes.

5 Lessons Learned

Planning ahead is important. For each lab in this course, we were required to submit a pre-lab plan, and the importance of this practice became even clearer in the final project. Our access to the robot was limited, so planning ahead helped ensure that everything we needed to examine could be covered during each session. This was something we did well. Each lab session was managed with an explicit schedule: what to test, how many minutes to allocate for each part, and what to do if something failed. This preparation made our time on hardware more efficient and reduced wasted runs.

Need to balance game strategy vs. chasing high-level technical design. For the dynamic part, we chose a complex approach: use vision to detect a moving block, predict its future position, and then approach that position by solving IK. This pipeline depends on precise vision outputs, a constant turntable speed, accurate prediction, and fast IK. In contrast, many other teams used a simpler method with a hard-coded waiting pose, letting the block move into the gripper. Our approach required managing more factors, and on hardware it demanded substantially more testing, modification, and debugging time, where a single weak link could change the outcome.

We discussed switching to the simpler hard-coded dynamic strategy, but decided to pursue the higher-level technical design to apply more of the methods learned in class. During the dress rehearsal and final competition, this decision became a bottleneck. Under the competition hardware conditions, the initial detection pose did not reliably return from the camera pipeline; detections only appeared when the block was lifted slightly higher by hand. In discussion with the TAs, we suspected that the reason is the newly attached AprilTag on the block was smaller than the tags used in the lab, which caused higher pose reduced detection reliability. With limited rehearsal time, we could not find a camera pose that worked consistently.

This experience highlighted a lesson we could apply more consistently: some groups achieved strong results with a simple hard-coded pose, and under this game’s constraints, the most effective and efficient approach can be better than the most technically ambitious one. This is also true in real-world manipulation projects: use what is most workable under the constraints, even if it is not the fanciest method. We made this trade-off in other parts of the system (e.g., not using RRT planning), but did not apply the same mindset when choosing the dynamic strategy. We still like our current implementation—making it work is a fun and meaningful process, and with more time it could be a strong choice. However, under the given time and rules, a key improvement would be to maintain a reliable backup solution: a hard-coded grasp pose for the dynamic stage.

Organize our code and do not be lazy. One lesson reinforced by the dynamic failure during the dress rehearsal and competition is the importance of consistently organizing our code and avoiding “quick” modifications without thinking about future changes. In our implementation, the dynamic detection pose was hard-coded as a robot joint configuration, while most other poses were stored as world-frame poses and converted to joint configurations via IK before each run. In hindsight, storing the dynamic detection pose as a world-frame pose would have been more flexible. We considered making this change, but did not, because the joint-configuration pose never failed during regular lab sessions.

During rehearsal and competition, the environment and hardware conditions changed, and the previously reliable joint-configuration pose no longer produced consistent detections. At that point, adjusting the pose in joint space was much harder than adjusting a world-frame pose (e.g., moving slightly higher or shifting laterally and re-solving IK). This lack of flexibility slowed down iteration and contributed to our inability to quickly find a detectable pose for the dynamic stage. Going forward, we should structure the code so that key behaviors are parameterized in world coordinates whenever possible, making last-minute tuning faster, safer, and less error-prone.

Debug is hard, but we can finally find a reason and solve the problem—and discussion with our community is helpful for this process. Besides the dynamic detection pose not returning reliable detections, we encountered several additional issues: IK failures on the Blue 2 robot due to targets outside the safe reachable workspace, occasional incorrect grasp heights in the dynamic stage, and missed detections in the static stage caused by worn or unclear AprilTags. Many of these problems looked “weird” at first, but became explainable once we isolated the root causes through systematic testing.

A key takeaway is that we consistently dug into problems to find both a fix and a root cause. This kind of debugging is common in real robotics projects: the robot never operates in a perfect world, and small environmental changes can trigger unexpected failures. Staying calm, being systematic, and “helping the robot out” are essential. We are also grateful for our community—discussions with the TAs and other groups were extremely helpful, often providing new hypotheses, debugging strategies, and practical insights that accelerated our progress.

Game result may vary, but when we enjoy the process, we gain and win. We continuously cared so much about our robot’s performance because we truly believed we had built a strong program and a solid strategy. Rather than treating this as a course assignment, we treated it as a real robotics project and pushed for better and better performance. However, during the dress rehearsal, when suddenly the dynamics did not work, we stopped being anxious about the outcome. Instead, we focused on our process and reflected on what we did well. We tried our best, we gained valuable experience and insights, and we enjoy it, so the result no longer mattered as much. We loved our team, enjoyed the project, and appreciated our robot even more (Figure 4).

A Appendix – Additional Figures



Figure 4: Robot Camera View at Initial Pose (When you look long into a robot, the robot looks into you.)

B Appendix – Pseudocode

ONESTEP

```

if not rospy.is_shutdown()
  switch self.state
    case INIT
      Open gripper
      goto SCAN
    case SCAN
      MoveTo(SCAN_CONFIG)
      blocks  $\leftarrow$  Vision.GetPoses()
      if blocks
        target  $\leftarrow$  blocks[0]
        goto MOVE_TO_PICK
      else
        goto DYNAMIC_INIT
    case MOVE_TO_PICK
      target.z  $\leftarrow$  config.PLATFORM_HEIGHT + offset
      grasp_pose  $\leftarrow$  Driver.CalculateGraspPose(target)
      if MoveTo(pre_grasp) and MoveTo(grasp_pose)
        goto GRASP
      else
        goto RECOVERY
    case GRASP
      Close gripper
      if Driver.CheckGraspSuccess()
        goto MOVE_TO_PLACE
      else
        Open gripper; goto SCAN
    case MOVE_TO_PLACE
      pose_drop  $\leftarrow$  CalculatePlacement(tower_height)
      if MoveTo(pose_drop)
        Release block; tower_height++
        goto SCAN
      else
        goto RECOVERY
    case DYNAMIC_MONITOR
      for each block in blocks do
        pred  $\leftarrow$  Driver.PredictPose(block,  $\Delta t$ )
        if pred.x in TRIGGER_ZONE
          target  $\leftarrow$  Driver.CalculateGraspPose(pred, dynamic)
          if MoveTo(target, dynamic)
            goto DYNAMIC_GRASP

```

Algorithm 1: PizzaHutFSM: High-Level Finite State Machine

```

PREDICTDYNAMICPOSE( $T_{block}$ ,  $dt$ )
   $P_c \leftarrow \text{TURNTABLE\_CENTER}$ ;  $\theta \leftarrow \omega \cdot dt$ 
   $R_{turn} \leftarrow \text{RotationMatrixZ}(\theta)$ 
   $P_{pred} \leftarrow P_c + R_{turn}(P_{old} - P_c)$ 
   $R_{pred} \leftarrow R_{turn} \cdot R_{old}$ 
  return [ $R_{pred}$ ,  $P_{pred}$ ]

CALCULATEGRASPOSE( $T_{block}$ ,  $mode$ )
   $\mathbf{z}_{target} \leftarrow [0, 0, -1]^T$ 
   $\mathbf{x}_{new} \leftarrow \mathbf{x}_{block} - (\mathbf{x}_{block} \cdot \mathbf{z}_{target})\mathbf{z}_{target}$ 
   $\mathbf{y}_{new} \leftarrow \mathbf{z}_{target} \times \mathbf{x}_{new}$ 
   $R_{final} \leftarrow [\mathbf{x}_{new}/|\mathbf{x}_{new}|, \mathbf{y}_{new}/|\mathbf{y}_{new}|, \mathbf{z}_{target}]$ 
  if  $mode = \text{dynamic}$ 
     $P_{final.z} \leftarrow P_{final.z} + \delta_{dyn}$ 
  return [ $R_{final}$ ,  $P_{final}$ ]

MOVETOPOSE( $T_{target}$ ,  $mode$ )
   $q_{target} \leftarrow \text{IK\_Solve}(T_{target}, q_{curr})$ 
  if IK success
    if  $mode = \text{dynamic}$ 
      Normalize  $q_{target}[6]$  to nearest  $\pm 90^\circ$ 
    arm.safe_move_to_position( $q_{target}$ )
    return True
  return False

```

Algorithm 2: Driver: Kinematics and Dynamic Prediction